

TER-Kai · Especificação Técnica e de Implementação

1 — Introdução, Propósito e Contexto

1.1 Objetivo do documento

Estabelecer as especificações técnicas da plataforma TER-Kai. O documento descreve a arquitetura de *middleware* para **governança, auditoria e correção** de sistemas baseados em LLMs (Large Language Models).

O objetivo é fornecer um *blueprint* implementável para um sistema que transforma saídas de LLM em respostas **verificáveis e seguras** para adoção corporativa e conformidade regulatória.

1.2 Tese central

O TER-Kai propõe uma arquitetura de *middleware* que torna as saídas de LLMs auditáveis, seguras e em conformidade com políticas corporativas.

O sistema opera sobre três pilares técnicos:

1. **Governança (\$\Omega\$):** Um **Motor de Regras (Policy Engine)** que intercepta respostas de LLM e aplica políticas de *compliance* (ex: detecção de PII, risco legal, diretrizes de tom) em tempo real.
2. **Auditoria (PoP-L):** Um **Ledger Criptográfico (Proof-of-Prudence Ledger)** que gera uma prova de integridade imutável (hash SHA3-512 + *Timestamping* TSA) para cada decisão tomada, garantindo rastreabilidade total para auditorias internas e externas (ex: AI Act, NIST).
3. **Correção (Reviser):** Um **Loop de Correção Automático** que é acionado quando uma resposta viola uma política (REVIEW). O sistema gera um *prompt* corretivo e utiliza o próprio LLM para gerar uma nova resposta segura, antes da exposição ao usuário final.

1.3 Problema técnico que resolve

1. **Alucinação convincente:** modelos geram respostas sem fontes robustas, com alta plausibilidade semântica e baixa veracidade.
2. **Falta de rastreabilidade:** decisões não são justificáveis nem reproduzíveis sob auditoria.
3. **Governança dispersa:** regras de segurança/ética ficam diluídas nos *prompts*, sem motor de políticas e métricas operacionais.
4. **Custo/latência de verificação:** checagem *claim-a-claim* sem priorização explode *tokens*.

1.4 Escopo

Coberto neste documento:

- Arquitetura e responsabilidades dos módulos (Kai, MIR, Ω , Retro, Ledger/PoP-L).
- **Definição das APIs de *Plugins*** para ingestão de métricas de análise (ex: sentimento, detecção de PII, verificação de bias).
- **Especificação do Motor de Regras (Ω)** e do formato de configuração PolicyConfig.json.
- **Estrutura de dados e fluxo criptográfico do Ledger (PoP-L)**, incluindo a integração com Autoridades de *Timestamping* (TSA).
- Fluxos operacionais (Fast/Deep Path; lógica do Reviser).
- Roteiro MVP e requisitos de implementação.

Fora de escopo (neste momento):

- **Desenvolvimento de métricas éticas proprietárias (ex: ECI, ϵ).**
(Nota: A arquitetura é projetada para *consumir* tais métricas via *plugins* quando elas estiverem disponíveis e validadas, mas não depende delas para operar.)
- AGI/ASI plena; firmware ético em hardware.

1.5 Público-alvo

Engenharia de IA, Segurança, Compliance e Auditoria (AI Act, NIST AI RMF), e Pesquisa aplicada.

1.6 Premissas e limitações (realismo 2025)

- O TER–Kai opera como *middleware* sobre LLMs; não substitui o modelo.
- **As métricas são consumidas, não inventadas:** O motor Ω depende de *plugins* de métricas (internos ou de terceiros) para tomar decisões. A qualidade da governança é diretamente proporcional à qualidade desses *plugins* de análise (sentimento, PII, bias, etc.).
- **O Ledger é prova de *decisão*, não de *verdade*:** O PoP-L prova, de forma irrefutável, que uma *política foi aplicada* a uma resposta em um determinado momento. Ele não atesta a veracidade absoluta da informação do LLM.
- Domínios sensíveis: HITL (Human-in-the-Loop) é obrigatório se o motor de regras Ω não puder garantir a conformidade com uma política crítica.

1.7 Conceitos nucleares (glossário)

- **Kai (L0): Extrator de Métricas Brutas.** Módulo que executa análises leves e extração de dados da resposta (ex: contagem de PII, extração de *claims* factuais para o *Deep Path*).
- **MIR (L1): Agregador de Métricas.** Coleta os dados do Kai e de *plugins* externos (ex: APIs de sentimento/bias) e os formata em um único pacote JSON (*metrics_input*) para o Ω .
- **Ω (L2): Motor de Governança.** Executa o PolicyConfig.json (as regras da empresa) contra o *metrics_input* e decide: ALLOW / REVIEW / BLOCK.

- **Retro (L3): Motor de Correção.** Gerencia o *rollback* e aciona o Reviser em caso de decisão "REVIEW".
- **Ledger / PoP-L (L4): Ledger de Auditoria Criptográfica.** Registra a prova (hash + TSA) da decisão do Ω e seu *rationale* (justificativa).
- **Reviser: Loop de Correção.** Gera uma nova resposta de LLM (com *prompt* corretivo) quando acionado pelo Retro.

TER-Kai · Especificação Técnica e de Implementação

2 — Arquitetura Geral e Estrutura de Governança

2.1 Visão macro da arquitetura

O TER-Kai é estruturado como um *middleware* de governança modular, posicionado entre o LLM (modelo base) e o usuário final (aplicação).

Seu objetivo é **interceptar, analisar, governar e auditar** cada resposta gerada. Ele não gera texto (exceto pelo Reviser), mas aplica um pipeline de conformidade e segurança ao texto gerado pelo LLM.

A arquitetura é composta por cinco blocos principais que operam em um pipeline sequencial e auditável.

Camada	Nome	Função Primária (v9.0)	Tipo
L0	Kai Core	Extração de Métricas Brutas e Fatos	Técnico-analítica
L1	MIR Reflexor	Agregação de Métricas (internas + <i>plugins</i>)	Técnico-analítica
L2	Ω / Meta- Ω	Motor de Governança (Aplicação de Políticas)	Ético-supervisora
L3	Retro Engine	Correção e Acionamento do Reviser	Prudencial-técnica
L4	Ledger / PoP-L	Registro e Auditoria Criptográfica	Auditoria / Criptográfica
—	Reviser (Kai-E)	Geração de Resposta Corretiva	Cognitivo / Corretivo

2.2 Estrutura conceitual

O funcionamento segue um fluxo de governança explícito, onde os dados são enriquecidos e verificados em cada etapa:

Usuário $\rightarrow$ LLM $\rightarrow$ [Kai (L0) $\rightarrow$ MIR (L1) $\rightarrow$ Ω (L2) $\rightarrow$ Retro (L3) $\rightarrow$ Ledger (L4)] $\rightarrow$ (Reviser) $\rightarrow$ Usuário

O bloco entre colchetes [] é o *pipeline* TER–Kai. Cada camada opera sobre pacotes de dados estruturados (JSON) e é projetada para ser independente e testável.

2.3 Fluxo operacional (pipeline)

1. Entrada da LLM (Texto Bruto):

O texto gerado pelo LLM chega ao Kai Core (L0).

2. Extração (Kai L0):

Kai "lê" o texto e extrai métricas brutas e fatos:

- Detecta PII (ex: CPF, e-mail, telefone, outros).
- Detecta palavras-chave de *compliance* (ex: "garantia", "diagnóstico").
- Extrai *claims* factuais (para o *Deep Path*).
- Produz o `kai_packet`.

3. Agregação (MIR L1):

MIR recebe o `kai_packet`. Sua função é enriquecê-lo.

- Consulta **APIs de Plugin** externas (ex: análise de sentimento, verificação de bias) ou internas.
- Agrega todos os dados em um único pacote, o `metrics_input`.

4. Governança (\$\Omega\$ L2):

\$\Omega\$ recebe o `metrics_input` e o `PolicyConfig.json` (as regras da empresa).

- Executa as regras (ex: if (`pii_count` > 0) or (`sentiment_score` < -0.8): return "BLOCK").
- Decide: **ALLOW / REVIEW / BLOCK**.

5. Correção (Retro L3):

- Se a decisão for **REVIEW**, Retro intercepta a resposta.
- Ele constrói um *prompt* corretivo e aciona o **Reviser**.

6. Auditoria (Ledger L4):

- A decisão final do \$\Omega\$ (ALLOW/REVIEW/BLOCK), o `metrics_input` e o *rationale* (justificativa da regra) são enviados ao **Ledger (PoP-L)**.
- O Ledger cria o registro criptográfico (hash SHA3-512 + TSA).

7. Resposta ao Usuário:

- Se ALLOW, o texto original é liberado.
 - Se REVIEW, o texto corrigido pelo Reviser é liberado.
 - Se BLOCK, uma mensagem de erro padrão é liberada.
-

2.4 Comunicação e pacotes de dados (APIs Internas)

As camadas se comunicam por pacotes JSON estruturados. Esta é a "planta" da nossa engenharia.

Pacote Kai $\rightarrow$ MIR (kai_packet):

Foco: Métricas extraídas do texto bruto.

```
{
  "raw_metrics": {
    "pii_detected": [
      {"type": "email", "value": "exemplo@email.com"}
    ],
    "pii_count": 1,
    "compliance_keywords": ["garantia"],
    "extracted_claims": [
      "O produto X é 50% mais rápido."
    ]
  },
  "timestamp": "2025-11-08T12:00:00Z"
}
```

Pacote MIR $\rightarrow$ Ω (metrics_input): Foco: Pacote agregado, pronto para o motor de regras.

```
{
  "aggregated_metrics": {
    "pii_count": 1,
    "compliance_keywords_found": ["garantia"],
    "plugin_sentiment_score": -0.85,
    "plugin_bias_score": 0.12
  },
  "claims": [
    "O produto X é 50% mais rápido."
  ],
  "timestamp": "2025-11-08T12:00:01Z"
}
```

TER–Kai · Especificação Técnica e de Implementação

Seção 3.1 — Módulos Internos e API de Governança (r3.1)

3.1 Visão Geral

O núcleo de processamento do TER–Kai é um pipeline de microsserviços que transforma texto bruto em uma decisão de governança auditável.

Cada módulo tem uma função discreta, transparente e testável: **extrair**, **agregar**, **governar**, **corrigir** e **auditar**.

Fluxo de dados:

LLM → Kai (L0) → MIR (L1) → Ω (L2) → Retro/Reviser (L3) → Ledger (L4)

3.2 Kai (L0) — Extrator de Métricas Brutas

Função: executar análise estática e extração leve de “sinais” óbvios.

Entrada: `input_text` (string)

Saída: `kai_packet` (JSON)

Principais rotinas →

Processo	Descrição	Aprimoramento r3.1
<code>detect_pii(text)</code>	regex + listas de padrões (CPF, e-mail, telefone).	Limite de <i>backtracking</i> e engine RE2 para segurança DoS.
<code>detect_keywords(text, policy_lists)</code>	procura palavras de risco (“garantia”, “diagnóstico”).	—
<code>extract_claims(text)</code>	identifica frases factuais para o Deep Path.	—
<code>basic_sentiment(text)</code>	análise léxica de sentimento.	—

3.3 MIR (L1) — Agregador de Métricas e Plugins

Função: orquestrar coleta e enriquecimento de dados.

Entrada: `kai_packet` (JSON) **Saída:** `metrics_input` (JSON)

```
let metrics_input = kai_packet.raw_metrics;

if (policy.plugins.includes("advanced_sentiment")) {
  const t0 = Date.now();
  metrics_input.plugin_sentiment_score =
    await SentimentPlugin.analyze(text);
  metrics_input.plugin_latency_ms = Date.now() - t0; // 🛠️ r3.1
}
```

```

if (policy.plugins.includes("bias_check")) {
  metrics_input.plugin_bias_score =
    await BiasPlugin.analyze(text);
}

// ... outros plugins
return metrics_input;

```

r3.1: Campo `plugin_latency_ms` adicionado → telemetria e observabilidade para MIR.

A arquitetura permanece **BYOM (Bring Your Own Metric)**, mantendo Ω agnóstico a métricas específicas.

3.4 Ω (L2) — Motor de Governança

Função: executar o `PolicyConfig.json` contra `metrics_input`.

Entrada: `metrics_input`, `PolicyConfig.json`

Saída: `decision_payload` (JSON) → ALLOW / REVIEW / BLOCK

r3.1: A sintaxe das condições segue agora o padrão **CEL (Common Expression Language)** ou **JSONLogic**, permitindo portabilidade entre motores open source.

```

{
  "domain": "financial_advice_chatbot",
  "version": 1.3,
  "rules": [
    {
      "name": "BLOCK: PII Detectado",
      "priority": 1,
      "condition": "metrics.pii_count > 0",
      "action": "BLOCK",
      "rationale": "Detecção de PII. Violação de privacidade."
    },
    {
      "name": "REVIEW: Sentimento Muito Negativo",
      "priority": 10,
      "condition": "metrics.plugin_sentiment_score < -0.85",
      "action": "REVIEW",
      "rationale": "Tom do LLM muito negativo. Acionar Reviser."
    },
    {
      "name": "ALLOW: Padrão",
      "priority": 100,
      "condition": "true",
      "action": "ALLOW",
      "rationale": "Conforme."
    }
  ]
}

```

3.5 Retro (L3) — Motor de Correção

Função: executar ações com base na decisão do Ω .

```

async function handleDecision(decision, text) {

```

```

if (decision.action === "ALLOW") return text;
if (decision.action === "REVIEW") {
  const prompt = Reviser.buildPrompt(text, decision.rationale);
  const revised_text = await InvokeLLM(prompt, {timeout: 4_000}); // 🔧 r3.1
  return revised_text;
}
if (decision.action === "BLOCK")
  return "Mensagem bloqueada por conformidade.";
}

```

r3.1: timeout 4 s → impede loops ou latência excessiva.

Todos os resultados (REVIEW) são re-hashados para Ledger L4.

3.6 Ledger / PoP-L (L4) — Ledger Criptográfico de Auditoria

Função: registrar decisões com prova imutável e timestamp certificado.

Aprimoramentos r3.1

- Adicionados campos `tsa_authority_uri`, `tsa_pubkey_fingerprint`.
- Inclusão obrigatória de `text_hash_output` e `revised_text_hash`.
- Sanitização automática de PII antes da serialização (`Kai.hashPII()`).

```

{
  "popl_id": "uuid-v4-...",
  "timestamp_request": "2025-11-08T12:00:03Z",
  "tsa_authority_uri": "https://tsa.example.org",
  "tsa_pubkey_fingerprint": "SHA256:AB34...",
  "policy_domain": "financial_advice_chatbot",
  "metrics_input": { "pii_count": 0, "plugin_sentiment_score": -0.87 },
  "final_decision": { "action": "REVIEW", "rule": "Sentimento Muito Negativo" },
  "text_hash_input": "sha256(input_text)",
  "text_hash_output": "sha256(revised_text)",
  "revised_text_hash": "sha256(revised_text)",
  "pii_sanitized": true
}

```

3.7 Segurança e Governança de Logs

: Todos os logs gerados pelo MIR e Ω devem passar por sanitização antes da persistência → `maskPII()` e `redactSensitiveFields()`.

3.8 Conclusão r3.1

Eleva o nível de **segurança, observabilidade, portabilidade e auditoria externa** TER–Kai ·
Especificação Técnica e de Implementação

Seção 4: Ciclo de Processamento e Fluxos Operacionais

4.1 Visão geral

O Ciclo de Processamento Prudencial (CPP) é o *pipeline* operacional que executa o TER–Kai. Ele define *como* e *quando* os módulos (Kai, MIR, Ω , Retro, Ledger) são acionados.

O sistema opera em dois modos principais para otimizar a relação custo (tokens, latência) vs. risco (compliance, segurança):

1. **Fast Path (Modo Rápido):** Triagem leve e de baixa latência, ideal para interações de conversação em tempo real.
2. **Deep Path (Modo Completo):** Análise profunda e de maior custo, reservada para domínios de alto risco, auditoria ou verificação de fatos.

O sistema utiliza um **Roteamento Baseado em Risco (RBR)** para decidir qual *path* utilizar.

4.2 Roteamento Baseado em Risco (RBR)

O TER–Kai não executa o *Deep Path* (caro) em todas as respostas. Antes do processamento completo, o sistema faz uma triagem inicial para determinar o nível de risco da interação.

- **Fatores de Risco:** O roteador avalia:
 1. **Risco de Domínio:** O PolicyConfig.json (Seção 3.4) define um `domain_risk` (ex: "Bate-papo geral" = 0.1, "Saúde" = 0.8).
 2. **Risco de Contexto:** Métricas do Kai (L0) (ex: detecção de `compliance_keywords`).
 3. **Risco de Severidade:** Se o usuário está em um fluxo de trabalho crítico (ex: "auditoria").
- **Lógica de Roteamento:**

```
if (domain_risk > 0.7) or (kai_packet.compliance_keywords_found.length > 0):
    mode = "DEEP_PATH"
else:
    mode = "FAST_PATH"
```

4.3 Fast Path (Modo Rápido)

Utilizado na maioria das interações (ex: 80-90% do tráfego) para garantir baixa latência.

- **Pipeline:** Kai (L0) $\rightarrow$ MIR (L1, *plugins* leves) $\rightarrow$ Ω (L2) $\rightarrow$ Retro (L3) $\rightarrow$ Ledger (L4, *Log Nível L1*).
- **Características:**

- **Kai (L0):** Executa apenas extrações rápidas (ex: PII, palavras-chave).
 - **MIR (L1):** Aciona apenas *plugins* de baixa latência (ex: análise de sentimento local). **Não** aciona *fact-checking* externo.
 - **\$\Omega\$ (L2):** Aplica o PolicyConfig.json sobre este conjunto limitado de métricas.
 - **Ledger (L4):** Gera um log L1 (local, sem TSA) para performance, marcando para *batching* de TSA posterior se necessário.
 - **Objetivo:** Garantir *compliance* básica (PII, linguagem de risco) com latência mínima.
 - **Latência Alvo:** \$\le\$ 800 ms (P95) adicionais à chamada do LLM.
-

4.4 Deep Path (Modo Completo)

Ativado automaticamente pelo RBR para domínios sensíveis ou quando *claims* factuais são detectados.

- **Pipeline:** Kai (L0) \$\rightarrow\$ MIR (L1, *plugins* completos) \$\rightarrow\$ \$\Omega\$ (L2) \$\rightarrow\$ Retro (L3) \$\rightarrow\$ Ledger (L4, *Log Nível L2*).
 - **Características:**
 - **Kai (L0):** Executa todas as extrações, incluindo `extract_claims()`.
 - **MIR (L1):** Aciona *plugins* pesados e de alto custo/latência:
 - **Verificação de Fatos (Fact-Checking):** O MIR recebe os `extracted_claims` do Kai. Para cada *claim*, ele usa um LLM (ou API de busca) para verificar a veracidade contra fontes confiáveis (Tier A/B).
 - **Análise de Biais:** Pode acionar modelos de análise de bias mais complexos.
 - **\$\Omega\$ (L2):** Aplica o PolicyConfig.json usando o conjunto de métricas completo (incluindo `fact_check_result`, `bias_score`, etc.).
 - **Ledger (L4):** Gera um log L2 completo (o "Dossiê" da Seção 3.6), que é **imediatamente** enviado para *timestamping* TSA, garantindo prova de auditoria.
 - **Latência Alvo:** 3–7 segundos (depende da velocidade do *fact-checking* externo).
-

4.5 Lógica do Módulo Reviser (Acionado pelo Retro)

Quando o \$\Omega\$ decide **REVIEW**, o Módulo Retro (L3) intercepta a resposta e aciona o Reviser. O Reviser é um processo de auto-correção que utiliza o próprio LLM.

- **Função:** Gerar uma nova resposta segura e em conformidade, baseada na falha detectada.

- **Processo:**

1. **Entrada:** input_text (texto original do LLM) e decision_rationale (da \$ \Omega\$, ex: "Tom do LLM muito negativo").
2. **Construção do Prompt Corretivo:** O Retro constrói um *meta-prompt* para o LLM.
3. **Execução:** InvokeLLM(correction_prompt).
4. **Nova Verificação:** A revised_text (resposta corrigida) é enviada de volta ao *pipeline* TER–Kai em modo *Fast Path* para garantir que a *nova* resposta também esteja em conformidade (ex: não introduziu PII).

- **Exemplos de Prompts Corretivos (Implementáveis):**

1. **Cenário (do \$ \Omega\$):** rationale: "Detecção de PII. Violação de privacidade."
2. **Prompt Corretivo:**

"A resposta a seguir contém informações privadas: [input_text]. Reescreva esta resposta para o usuário, mantendo o sentido original, mas **removendo todas as informações de identificação pessoal (PII)**, como e-mails, telefones ou nomes."
3. **Cenário (do \$ \Omega\$):** rationale: "Tom do LLM muito negativo. Acionar Reviser."
4. **Prompt Corretivo:**

"A resposta a seguir foi considerada muito negativa ou antipática: [input_text]. Reescreva esta resposta para o usuário, mantendo a informação factual, mas ajustando o tom para ser **profissional, neutro e prestativo**."

4.6 Pseudocódigo Completo do Ciclo

Este pseudocódigo TypeScript ilustra o *pipeline* de decisão.

```
// O backend orquestra esse fluxo
async function handleLLMResponse(llm_text, context, policy) {

  // 1. Extração (L0)
  const kai_packet = Kai.extract(llm_text, policy);

  // 2. Roteamento (RBR)
  const mode = RiskRouter.getMode(kai_packet, policy.domain_risk);

  // 3. Agregação (L1)
  // O MIR usa o 'mode' para decidir quais plugins chamar (leves ou pesados)
  const metrics_input = await MIR.aggregate(kai_packet, policy, mode);
```

```

// 4. Governança (L2)
const decision_payload = Omega.govern(metrics_input, policy.rules);

// 5. Correção (L3)
let final_text;
if (decision_payload.action === "REVIEW") {
  final_text = await Retro.correct(llm_text, decision_payload.rationale);

  // Nova verificação rápida da resposta corrigida
  const final_kai_packet = Kai.extract(final_text, policy);
  const final_metrics = await MIR.aggregate(final_kai_packet, policy, "FAST_PATH");
  const final_decision = Omega.govern(final_metrics, policy.rules);

  if (final_decision.action !== "ALLOW") {
    // Se a correção falhar, bloqueia
    final_text = "Mensagem bloqueada por conformidade.";
  }
} else if (decision_payload.action === "BLOCK") {
  final_text = "Mensagem bloqueada por conformidade.";
} else {
  final_text = llm_text; // ALLOW
}

// 6. Auditoria (L4)
// O Ledger registra o 'metrics_input', a 'decision_payload',
// e o 'mode' (Fast/Deep) e se foi 'revised'.
await Ledger.log(metrics_input, decision_payload, final_text, mode);

return final_text;
}

```

Aprioramentos

Área	Pontos positivos	Ajuste sugerido	Impacto
Fast / Deep Path	Separação clara entre custo e risco, com latência plausível.	Incluir um “fallback automático” : se Deep Path falhar por timeout ou erro de API externa, o sistema deve recuar para Fast Path + flag "partial_audit": true no Ledger.	Resiliência
RBR (RiskRouter)	Lógica clara e testável.	Adicionar fator "history_risk" (detecção de reincidência de termos críticos por usuário).	Governança longitudinal
Reviser	Prompts bem estruturados, realistas e neutros.	Registrar no Ledger: "revision_count" e "prompt_type" (ex.: tone_correction , pii_removal).	Auditabilidade
Ledger (L4)	Log L1/L2 alinhado a PoP-L.	Padronizar os níveis de log como "level": "L1" ou "L2" no schema.	Clareza
Pseudocódigo	Estrutura próxima	Acrescentar try/catch global para	Confiabilidade

Área	Pontos positivos	Ajuste sugerido	Impacto
Compliance	da execução real	capturar exceções de plugin e enviar alerta ao Ω Audit.	
	Alinhado a AI Act/NIST RMF.	Mencionar explicitamente LGPD/GDPR no trecho de PII e logging.	Conformidade jurídica

TER–Kai · Especificação Técnica e de Implementação

Seção 5: Conformidade, Segurança e Integridade da Auditoria (r5.1)

5.1 Visão Geral

Esta seção detalha os mecanismos de confiança da arquitetura TER–Kai. Os módulos definidos anteriormente (\$\Omega\$, PoP-L, Kai, Reviser) não são apenas funcionais; eles formam um "esqueleto de confiança" coeso.

O objetivo deste esqueleto é fornecer garantias técnicas, legais e éticas, permitindo que a organização prove o que aconteceu, por que aconteceu e como foi corrigido. Os pilares são: **Integridade da Auditoria**, **Conformidade Regulatória** e **Segurança de Dados**.

5.2 Pilar 1: Integridade da Auditoria (O Ledger PoP-L)

O Módulo Ledger / PoP-L (definido na Seção 3.6 r3.1) é o pilar central da integridade.

- **Não-Repúdio (O Valor de Negócio):** O valor do PoP-L não é apenas *registrar*, mas garantir o *não-repúdio*. O processo de *hash* (SHA3-512) + *Timestamping* (TSA RFC 3161) cria uma **prova criptográfica de existência**.
- **O Dossiê de Decisão (DED):** Cada registro PoP-L é um "Dossiê de Decisão" completo. Ele prova que *exatamente aquele metrics_input* foi avaliado pela *exatamente aquela policy_version* e resultou na *exatamente aquela decision*.
- **Rastreabilidade de Fila (TSA):** O DED deve incluir um campo de *status* ("tsa_status": "pending | confirmed") para gerenciar a fila de *timestamping*. Uma decisão só é considerada imutável quando o *status* é "confirmed".
- **Auditabilidade:** O DED é o artefato entregue a um auditor. Qualquer tentativa de alterar o *metrics_input* ou a *decision* no banco de dados invalidaria o *hash* e seria imediatamente detectada pela verificação do *tsa_token*.

5.3 Pilar 2: Conformidade Regulatória (Mapeamento)

A arquitetura v9.0 foi projetada para fornecer conformidade "de fábrica" com as principais regulações globais de IA.

Framework Regulatório	Recurso do TER–Kai	Como ele atende à exigência
AI Act (UE)	Ledger / PoP-L (Art. 17)	Atende diretamente à exigência de "logging" automático, imutável e rastreável para sistemas

Framework Regulatório	Recurso do TER–Kai	Como ele atende à exigência
		de alto risco.
AI Act (UE)	\$\Omega\$ (Art. 13) / Reviser (Art. 15)	Fornece transparência (o <i>rationale</i> no \$\Omega\$) e capacidade de supervisão/correção humana (ganchos de HITL e o Reviser).
NIST AI RMF	\$\Omega\$ / Ledger PoP-L	Implementa os princípios de " Govern " (Políticas \$\Omega\$), " Measure " (Métricas do MIR) e " Manage " (Reviser e Logs PoP-L).
LGPD / GDPR	Kai (L0) / \$\Omega\$ (L2)	O Kai (com detect_pii) e o \$\Omega\$ (com regras BLOCK if pii_count > 0) implementam ativamente a Privacidade por Design (Privacy by Design) .
ISO 42001	Arquitetura v9.0 (Completa)	Fornece o " Sistema de Gestão de IA " técnico (o "como") necessário para que as organizações obtenham a certificação ISO, alinhando-se aos controles exigidos.

5.4 Pilar 3: Segurança e Privacidade de Dados

A governança de IA falha se o próprio sistema de governança se torna um risco de privacidade.

- **Redação (Redaction) e Minimização de Dados no Ledger:** Conforme definido na Seção 3.6 (r3.1), o PoP-L foi projetado para **não armazenar o texto bruto (PII) por padrão** (o campo "pii_sanitized": true). Ele armazena metadados e *hashes*, permitindo uma auditoria completa **sem** criar um "vazamento de dados" (honeypot) no próprio banco de logs.
- **Controle de Acesso Baseado em Função (RBAC):** A plataforma TER–Kai deve impor RBAC estrito:
 - **Engenheiros:** Podem manter os *plugins* e o motor \$\Omega\$.
 - **Equipe de Compliance:** Pode ler e escrever as **regras** (PolicyConfig.json).
 - **Audidores (HITL):** Podem visualizar os **Dossiês de Decisão (DED)** e as respostas marcadas como REVIEW.
- **Criptografia e Gerenciamento de Chaves:** A política de segurança de dados do PoP-L deve ser explicitamente:
 - **Criptografia em Repouso:** encryption_at_rest: AES-256-GCM.
 - **Gerenciamento de Chaves:** Uso obrigatório de um KMS (Key Management Service).
 - **Política de Rotação:** key_rotation: 90 dias (ou conforme política de *compliance* do cliente).

5.5 Tratamento de Incidentes e Resiliência

Um sistema de governança deve ser resiliente a falhas em suas dependências (ex: os *plugins*).

- **Política de Retentativa (Retry Policy):** Todas as chamadas de rede externas (MIR para *plugins*, Ledger para TSA) devem aderir a uma política de *retry* padronizada:
 - **Estratégia:** Recuo Exponencial (Exponential Backoff).
 - **Configuração Padrão:** max_retries: 3, backoff_multiplier: 2x.
- **Cenário 1: Falha no *Plugin* (ex: API de Sentimento).**
 - **Solução:** O PolicyConfig.json (do Ω) deve incluir regras de "fail-safe" (ex: REVIEW se a métrica falhar).
- **Cenário 2: Falha no Reviser.**
 - **Solução:** Conforme o pseudocódigo (Seção 4.6), o sistema deve "travar" em segurança, resultando em BLOCK. A resposta falha *nunca* é exposta.
- **Cenário 3: Falha no *Timestamping* (TSA).**
 - **Solução:** Após exceder os *retries*, a transação é marcada no DED como "tsa_status": "pending" e colocada em uma fila de retentativa de longo prazo. A decisão (ex: BLOCK) é aplicada, mas o log PoP-L só é "confirmed" (imutável) após o sucesso da TSA.

5.6 Entregáveis de Auditoria (Os Relatórios)

A arquitetura v9.0 produz dois artefatos de auditoria principais:

1. **Dossiê de Decisão (DED):** (O "Recibo") O arquivo JSON + TSA para uma *única transação*, detalhando as métricas, a regra acionada e o *hash* do resultado.
2. **Relatório de Atividade Prudencial (RAP):** (O "Dashboard") Um relatório agregado que mostra tendências de alto nível para a gerência de risco. Ex:
 - "% de respostas Bloqueadas/Revisadas por domínio."
 - "Principais regras do Ω acionadas."
 - "Volume de detecção de PII."
 - **Métrica de Performance:** "Média de latência por *path* (Fast/Deep)".

TER-Kai · Especificação Técnica e de Implementação

Seção 6: Testes, Validação (V&V) e Métricas de Sucesso (r6.1)

6.1 Visão Geral

Esta seção define a estratégia de **Validação e Verificação (V&V)** para a plataforma TER-Kai. O objetivo é duplo:

1. **Verificação (Engenharia):** Provar que os módulos de *software* (Kai, Ω , Ledger) funcionam conforme a especificação técnica (Seção 3).
2. **Validação (Compliance):** Provar que o sistema, como um todo, mitiga efetivamente os riscos de negócio (Seção 6.3) e é preciso (Seção 6.4).

O sucesso do MVP será medido pela capacidade demonstrável de **reduzir riscos de forma precisa, aplicar políticas e fornecer provas de auditoria robustas**.

6.2 Estratégia de Testes (Engenharia)

Refere-se aos testes de *software* para garantir que os módulos sejam robustos, seguros e resilientes.

- **Testes de Unidade (Unit Tests):**
 - **Módulo Kai (L0):**
 - Testar `detect_pii()` com dados de amostra (`Assert pii_count`).
 - **(r6.1) Adicionar Testes de Fuzzing** leves ao `detect_pii()` para garantir que entradas maliciosas ou inesperadas não causem falhas (ex: ReDoS) e confirmem a sanitização.
 - **Módulo Ω (L2):** Testar o motor de regras com *mocks* de `PolicyConfig.json` e `metrics_input`, afirmando (`Assert`) que as decisões e *rationales* corretos são retornados.
 - **Módulo Ledger (L4):** Testar a serialização do Dossiê e a geração correta do `payload_hash` (SHA3-512).
 - **Testes de Integração (Integration Tests):**
 - **Pipeline E2E:** Testar o *pipeline* completo (Seção 4.6), afirmando (`Assert`) que um *prompt* com PII resulta em BLOCK.
 - **Loop do Reviser:** Testar que um *prompt* REVIEW aciona corretamente a segunda chamada `InvokeLLM`.
 - **(r6.1) Teste de Resiliência de Falha:** Incluir um teste de integração que **simula uma falha na TSA** (erro 500). O teste deve afirmar (`Assert`) que o `retry-policy` (Seção 5.5) é acionado e o Dossiê é marcado com `"tsa_status": "pending"` no banco de dados.
-

6.3 Validação de Eficácia (Compliance e QA)

Refere-se à validação de que as políticas de negócio são aplicadas corretamente.

- **Banco de Cenários de Risco (Golden Set):**

- A equipe de *compliance* (não a engenharia) deve criar um "banco de cenários" (*golden_set.json*) com *prompts* que simulam riscos reais.
- O processo de V&V consiste em rodar esses *prompts* contra o sistema e verificar se o **resultado observado** (a decisão do Ω e o log PoP-L) corresponde ao **resultado esperado**.
- **(r6.1) Rastreabilidade da Validação:** Para auditorias externas (ISO, NIST), o próprio *golden_set_v1.0.json* (e suas versões futuras) deve ser **registrado e assinado no PoP-L**. Isso cria um rastro de auditoria *do próprio processo de teste*, provando quais cenários estavam sendo validados em qual data.
- **Exemplos de Cenários de Validação (V&V):**

ID do Teste	Tipo de Risco	Resultado Esperado (Validado)
V&V-001	Privacidade (PII)	BLOCK. Log PoP-L deve citar rationale: "Detecção de PII."
V&V-002	Risco Legal	BLOCK. Log PoP-L deve citar rationale: "Uso não autorizado de 'garantia'."
V&V-003	Risco de Tom	REVIEW. A <i>final_text</i> deve ser a <i>resposta corrigida</i> pelo Reviser.
V&V-004	Falso Positivo	(Prompt limpo, sem risco)

6.4 Métricas de Sucesso (KPIs) do MVP

Refere-se às métricas de alto nível que provam o **valor de negócio** e a **eficácia operacional** do MVP.

- **Eficácia de Risco (Segurança e Precisão):**

- **KPI 1:** 99.9% de detecção e bloqueio de violações de PII e palavras-chave de risco (conforme V&V-001, 002).
- **(r6.1) KPI 2:** Taxa de Falsos Positivos (FPR) $\leq 1\%$ (conforme V&V-004). O sistema deve ser seguro, mas não inutilizável.
- **Rastreabilidade (Auditoria):**
 - **KPI 3:** 100% das decisões de governança geram um Dossiê de Decisão (DED) no Ledger.

- **KPI 4:** 100% dos Dossiês do *Deep Path* obtêm um *tsa_token* válido e verificável (ou são enfileirados com *status* "pending").
- **Eficiência Operacional (Automação):**
 - **KPI 5:** O Módulo Reviser (auto-correção) resolve >80% das violações de REVIEW (ex: tom) sem exigir escalação para HITL.
- **Performance e Custo (Operações):**
 - **KPI 6:** A latência adicional do *Fast Path* (P95) permanece abaixo de 800ms.
 - **(r6.1) KPI 7:** O consumo médio de *tokens* (custo) por *path* (Fast/Deep/Reviser) é monitorado e reportado no RAP (Relatório de Atividade Prudencial).

6.5 (r6.1) Escalabilidade da QA e Transparência Corporativa

O processo de V&V deve ser escalável e transparente para auditorias externas.

- **Relatório de Validação Automatizado:** O sistema deve ter a capacidade de gerar automaticamente o **Relatório de V&V** (o resultado da execução do *Golden Set*).
- **Entregável de *Compliance*:** Este relatório deve ser exportável como um **PDF/A (padrão ISO de arquivamento) assinado** (via **PoP-L**), criando um artefato de *compliance* formal e imutável que pode ser entregue a reguladores (ex: AI Act) ou auditores (ex: ISO 42001).

TER–Kai · Especificação Técnica e de Implementação

Seção 7: Especificações Criptográficas e Algoritmos-Chave

7.1 Propósito da Seção

Esta seção consolida os algoritmos de engenharia, segurança e resiliência que formam o núcleo de implementação da arquitetura TER–Kai.

O objetivo é servir como referência técnica para engenheiros de segurança, arquitetos de software e equipes de SRE (Site Reliability Engineering) responsáveis pela implementação do *blueprint*.

7.2 Algoritmo 1: Geração de Prova Imutável (Ledger PoP-L)

- **Propósito (Valor de Negócio):** Garantir o **não-repúdio** de uma decisão de governança. Este processo cria uma prova verificável e com data/hora certificada por um terceiro (TSA), atendendo aos requisitos de auditoria mais rigorosos (ex: AI Act, Art. 17).
- **Especificação (Engenharia):** O processo de geração de log L4 (*Deep Path*) deve seguir este fluxo criptográfico:
 1. Serialização: O "Dossiê de Decisão" (DED), conforme definido na Seção 3.6 (r3.1), é serializado em uma string JSON determinística.

```
ded_string = JSON.stringify(Dossie)
```

2. Hashing: O hash do Dossiê é calculado usando um algoritmo resistente a colisões (SHA-3 é preferido; SHA-256 é o mínimo aceitável).

```
payload_hash = SHA3-512(ded_string)
```

3. Timestamping (TSA): O `payload_hash` é enviado a uma Autoridade de Timestamping (TSA) externa, compatível com o padrão RFC 3161.

```
tsa_token = await TSARequester.getTimestamp(payload_hash)
```

4. **Armazenamento:** O DED (JSON) e o `tsa_token` (binário/string) são armazenados no *ledger* de auditoria. O DED é marcado com `"tsa_status": "confirmed"`.

- **Justificativa Técnica:**

1. SHA3-512 garante a **Integridade do Conteúdo**: prova que o Dossiê (as métricas e a decisão) não foi alterado *após* o *hash* ser calculado.
2. TSA (RFC 3161) garante a **Integridade Temporal**: prova que o Dossiê *existia antes* da data/hora assinada pela TSA.

7.3 Algoritmo 2: Lógica de Regras Segura (Motor Ω)

- **Propósito (Valor de Negócio):** Permitir que a equipe de *compliance* (não-engenheiros) defina regras de negócio complexas de forma segura, sem criar vetores de ataque por injeção de código.
- **Especificação (Engenharia):** O campo `condition` no `PolicyConfig.json` (Seção 3.4, r3.1) **não deve** ser executado usando `eval()` ou interpretadores de linguagem nativos. A implementação do Ω deve usar um *parser* de lógica *sandboxed* e padrão de indústria.
- **Padrão Recomendado:** **CEL (Common Expression Language)** ou **JSONLogic**.
- Exemplo de "Fórmula" (CEL):

```
"condition": "metrics.pii_count > 0 && metrics.plugin_sentiment_score < -0.5"
```
- **Justificativa Técnica:** Motores como CEL (usado pelo Google) ou JSONLogic são *sandboxed* por design. Eles são **não-Turing completos** (não podem fazer loops) e **não têm acesso** a I/O (rede ou disco), impedindo que uma regra de política mal configurada execute código malicioso ou cause um ataque DoS.

7.4 Algoritmo 3: Política de Resiliência (Exponential Backoff)

- **Propósito (Valor de Negócio):** Garantir a resiliência do sistema contra falhas transientes de serviços externos (ex: *plugins* do MIR, TSA) sem causar sobrecarga (thundering herd).

- **Especificação (Engenharia):** Todas as chamadas de rede externas (Seção 5.5, r5.1) devem implementar uma política de retentativa (retry) com recuo exponencial e *jitter*.
- **"Fórmula" de Exemplo (Pseudocódigo):**

```
max_retries = 3
base_delay = 100 // milissegundos
```

```
// (2 ** retry_count) é o recuo exponencial
// (Math.random()) é o jitter (para evitar colisões)
delay_ms = (base_delay * (2 ** retry_count)) + (Math.random() * base_delay)
```

Justificativa Técnica: O recuo exponencial ($2^{**} \text{retry_count}$) impede que o *pipeline* sobrecarregue um serviço que já está falhando. O *jitter* (aleatoriedade) impede que múltiplas instâncias do TER-Kai tentem novamente exatamente ao mesmo tempo, distribuindo a carga.

7.5 Algoritmo 4: Mitigação de ReDoS (Extrator Kai)

- **Propósito (Valor de Negócio):** Proteger o *pipeline* de governança contra ataques de Negação de Serviço (DoS) que exploram a complexidade da detecção de PII.
 - **Especificação (Engenharia):** A função `detect_pii()` (Seção 3.2, r3.1), que depende de Expressões Regulares (Regex), **deve** utilizar um motor de Regex que garanta tempo de execução linear.
 - **Padrão Recomendado:** RE2 (ou similar, como o motor do Rust).
 - **Justificativa Técnica:** Motores de Regex tradicionais (ex: PCRE, usado em Python, JS, Java) são vulneráveis a "backtracking catastrófico" (ReDoS), onde uma entrada maliciosa pode travar a CPU por segundos ou minutos. O RE2 (usado pelo Google) sacrifica alguns recursos de *backtracking* pela garantia matemática de tempo de execução linear, o que é um requisito não-negociável para um *middleware* de segurança.
-

7.6 Algoritmo 5: Telemetria de Performance (Observabilidade)

- **Propósito (Valor de Negócio):** Fornecer a métrica de engenharia (KPI) básica para o monitoramento de performance (SRE) e otimização de custos (FinOps) dos *plugins* do MIR.
- **Especificação (Engenharia):** O Módulo MIR (Seção 3.3, r3.1) deve cronometrar cada chamada de *plugin* externo.
- **"Fórmula" de Exemplo (Pseudocódigo JS):**

```
const t0 = performance.now(); // Inicia o timer
const result = await SentimentPlugin.analyze(text);
const latency_ms = performance.now() - t0; // Finaliza o timer

metrics_input.plugin_sentiment_score = result;
metrics_input.plugin_sentiment_latency_ms = latency_ms;
```

- **Justificativa Técnica:** Esta métrica (`latency_ms`) é a fonte de dados primária para os KPIs de performance (Seção 6.4) e para os alertas de SRE (ex: "P99 da API de Sentimento > 800ms"), permitindo que a operação identifique gargalos de latência.